

JavaScript Currying: Cheat Sheet

Basics of Currying

- Currying is converting a function with multiple arguments into a series of unary functions.
- Example: $f(a, b, c) \rightarrow f(a)(b)(c)$
- Enables function reusability and partial application.

Manual Currying Example

- ```
function add(a) { return function(b) { return function(c) {
 return a + b + c; }; }; };
```
- Usage: `add(1)(2)(3) // 6`

## Generic Curry Utility

- ```
function curry(fn) {  
  return function curried(...args) {  
    if (args.length >= fn.length) return fn(...args);  
    else return (...next) => curried(...args, ...next);  
  };  
}
```

Use Cases

- Event Handlers: predefine arguments for DOM or React handlers.
- Middleware: chainable and configurable logic (e.g., Redux, Express).
- Functional Composition: reusable utilities with fixed arguments.

Currying vs Partial Application

- Currying: breaks down function to unary chain ($a \rightarrow b \rightarrow c$).
- Partial Application: pre-fills some arguments ($f(a, _, c)$).

Advanced Concepts

- Infinite Curry: `sum(1)(2)(3)...()` using closures and lazy evaluation.
- TypeScript: curry with generics and overloads for type safety.
- Uncurrying: flatten curried function to accept all args at once.

Pitfalls

- Deep closures can cause memory retention.
- Over-currying can reduce readability if not managed well.

Important Interview Questions

- What is currying in JavaScript and why is it useful?
- How does currying differ from partial application?
- Write a curried version of a sum function.
- Explain how currying helps in functional programming.
- What are some real-world use cases of currying in front-end development?
- Can you implement a generic curry utility function?
- What happens if you provide fewer arguments than expected in a curried function?
- How can currying help in composing higher-order functions?
- What are the potential downsides or limitations of using currying?
- How would you handle optional arguments or default values in curried functions?